

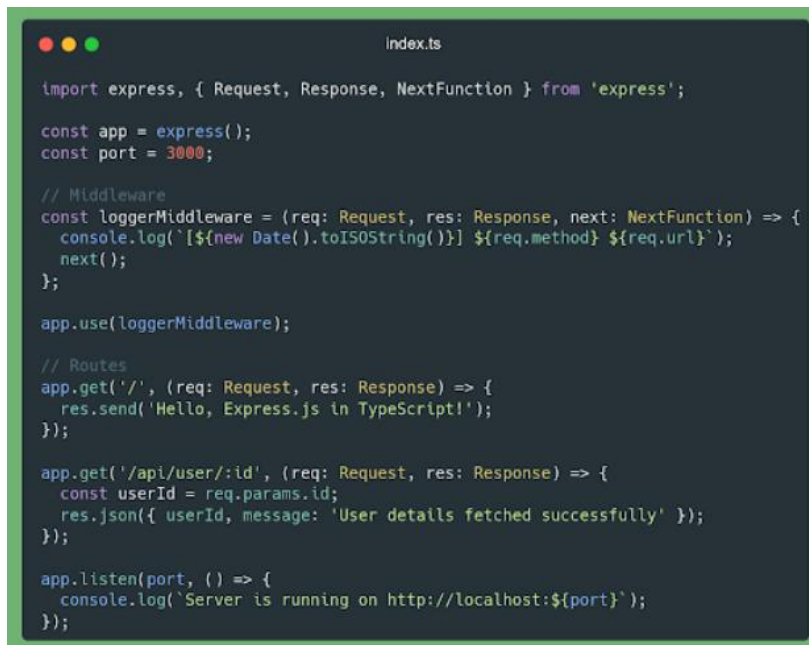
# Backend Quiz

JCWD-3302

1. What is the role of npm in Node JS application development?
  - a. **Creating the layout of a website.**
  - b. **Handling HTTP request**
  - c. **Managing dependencies and packages**
  - d. **To store data in a database**
2. How do you create a GET route endpoint in Express JS following proper REST conventions? (ex: for fetching a collection of products).
  - a. **app.get('/get-product', callback)**
  - b. **app.get('/products', callback)**
  - c. **app.post('/products', callback)**
  - d. **app.get('/product', callback)**
3. What is the difference between PUT and POST in the context of REST API?
  - a. **middlewareFunction.configure(app)**
  - b. **app.listen(middlewareFunction)**
  - c. **app.configure(middlewareFunction)**
  - d. **app.use(middlewareFunction)**
4. What is middleware in Express JS and how can you implement it to validate input data before saving it to the database?
  - a. **Middleware is the standard method for managing routes in Express JS**
  - b. **Middleware is a function that can modify the request or response object.**
  - c. **Middleware is used only to connect Express JS application with the database server.**
  - d. **Middleware can only be used for user session management.**
5. What is the difference between PUT and POST in the context of REST API?
  - a. **PUT and POST can both be used to update data.**
  - b. **PUT and POST can both be used to create new data**
  - c. **PUT is used to update data, while POST is used to create new data.**
  - d. **PUT is used to create new data, while POST is used to update data.**
6. The following is not for handling user authentication in REST API development:
  - a. **JSON Web Token (JWT)**
  - b. **Basic Authentication**
  - c. **OAuth 6.0**
  - d. **All answer are correct.**
7. How can you optimize the performance of a node js application handling high HTTP traffic?
  - a. **Use serverless computing techniques to manage high traffic loads and ensure application availability.**
  - b. **Avoid using clustering as it can cause unnecessary overhead and focus on optimizing the source code.**
  - c. **Increase the use of synchronous programming to ensure data consistency, and use monitoring tools to detect bottlenecks.**
  - d. **Use caching for frequently used query results and consider using a reverse proxy.**
8. Explain the concept of JWT (JSON Web Token) and how it is used in authentication and authorization in node.js application
  - a. **JWT is an authorization protocol used to verify user identity. Its usage involves exchanging tokens between the server and the user.**
  - b. **JWT is an authentication mechanism that provides authentication workflow within the server. Its usage involves storing tokens in cookies on both the server and the user client.**
  - c. **JWT is a structured token format that can be used to transmit user information. Its usage involves creating tokens and verifying them on each request.**
  - d. **JWT can only be used for authentication, and cannot be used for authorization. Its usage involves exchanging tokens between the server and the client.**

9. Explain the difference between authentication and authorization in the context of creating REST API
- Authentication is the process of granting access rights to a user, while authorization is the process of verifying the user's identity**
  - Authentication is the process of verifying a user identity, while authorization is the process of granting access rights based on that verified identity.**
  - Authentication and authorization are terms that can be used interchangeable because they have similar meanings.**
  - Authentication and authorization are concepts used to store data in cache on the server.**
10. How to create a new table with the condition of two columns where the first column is used to store numbers and the second column is used to store a name in PostgreSQL?
- CREATE new\_table (column1 INT, column2 VARCHAR(255));**
  - CREATE new\_table ADD COLUMN (column1 VARCHAR(255), column2 INT);**
  - CREATE TABLE new\_table (column1 VARCHAR(255), column2 INT);**
  - CREATE TABLE new\_table (column1 INT, column2 VARCHAR(255));**
11. What is meant by a Foreign Key (FK) in SQL and how do you use it?
- Foreign Key is used to sort data in a table based on a specific column.**
  - Foreign Key is used to identify unique records in a table.**
  - Foreign Key is used to link two tables together based on a specific column.**
  - Foreign Key is used to provide default values for certain columns.**
12. How do you delete all data from a table in PostgreSQL without deleting the table structure itself?
- TRUNCATE table\_name;**
  - DROP table\_name**
  - REMOVE FROM table\_name;**
  - CLEAR \* FROM table\_name;**
13. Explain the concept of normalization in the database design and why it is important?
- Normalization is not important in database design because it can increase the complexity of SQL queries.**
  - Normalization is the process of converting data into a simpler form. It is important to facilitate data readability in the database.**
  - Normalization is the process of grouping data in a database based on its data type. It is important to ensure efficient data storage.**
  - Normalization is the process of reducing redundancy and data dependencies in a database. It is important to ensure consistent data and reduce the risk of anomalies.**
14. What is a nested subquery and how can you use it in SQL?
- A nested subquery is a type of join operation that merges results from two tables.**
  - A nested subquery is a query within another query, used to provide intermediate results for the outer query.**
  - A nested subquery is a database trigger that automatically executes based on certain conditions.**
  - A nested subquery is a type of stored procedure that can be called from within a function.**
15. What is the purpose of the Prisma Migrate tool?
- To seed the database with random data**
  - To test database queries**
  - To generate and apply database schema changes based on the Prisma schema**
  - To update environment variables**
16. Which of the following data types is not supported by Prisma in the schema.prisma file?
- String**
  - Datetime**
  - Float64**
  - Enum**
17. What is meant by mocking in the context of unit testing in Express JS?
- Mocking is a step in the optimization process of Express JS to improve server response time.**
  - Mocking is a technique to replace external components with fake implementations to isolate the unit being tested.**

- c. **Mocking is the process of creating a replica of an Express JS server to replace the original server during unit testing.**
  - d. **Mocking is the process of creating identical data copies for testing purposes.**
18. What is the difference between `beforeEach` and `before` in unit testing with Jest in Express JS?
- a. **`before` is used to run a block of code before each test is executed, while `beforeEach` runs the code block only once before all tests are executed.**
  - b. `beforeEach` is used to configure the setup before each test, while `before` is used to configure the setup after each test.
  - c. `beforeEach` is used to run a block of code before each test is executed, while `before` runs the code block only once before all tests are executed.
  - d. `beforeEach` and `before` are the same, so they can be used interchangeably.
19. Consider the TypeScript code above that uses Express JS for routing and middleware. What does the `loggerMiddleware` function do in this Express JS application?



```
index.ts

import express, { Request, Response, NextFunction } from 'express';

const app = express();
const port = 3000;

// Middleware
const loggerMiddleware = (req: Request, res: Response, next: NextFunction) => {
  console.log(`${new Date().toISOString()} ${req.method} ${req.url}`);
  next();
};

app.use(loggerMiddleware);

// Routes
app.get('/', (req: Request, res: Response) => {
  res.send('Hello, Express.js in TypeScript!');
});

app.get('/api/user/:id', (req: Request, res: Response) => {
  const userId = req.params.id;
  res.json({ userId, message: 'User details fetched successfully' });
});

app.listen(port, () => {
  console.log(`Server is running on http://localhost:${port}`);
});
```

- a. **logs the timestamp, HTTP method, and URL of every incoming request to a file.**
  - b. **logs the timestamp, HTTP method, and URL of every outgoing response to the console.**
  - c. **logs the timestamp, HTTP method, and URL of every incoming request to the console.**
  - d. **handles authentication for protected routes using middleware.**
20. If we successfully add a new record to our database, which HTTP response code is most appropriate to use? If we do not have proper access to access a request, what response code should be provided?
- a. **200 & 401**
  - b. **201 & 500**
  - c. **201 & 401**
  - d. **200 & 500**

21. What will the following Prisma query return?

```
const users = await prisma.user.findMany({
  where: {
    email: {
      endsWith: "@example.com",
    },
  },
});
```

- a. All users whose email ends with "@example.com"
  - b. All users with verified emails
  - c. All users with empty emails
  - d. Syntax error due to invalid filter
22. Consider the following Typescript code that uses Express JS, Typescript, and Prisma to fetch data from the products and reviews tables. What does `parseInt(req.params.id, 10)`; do on the line `const productId = parseInt(req.params.id, 10)`; and what will happen if there is an error fetching data from the reviews table?)

```
app.get('/api/products/:id', async (req: Request, res: Response) => {
  try {
    const productId = parseInt(req.params.id, 10);
    const product = await prisma.product.findUnique({ where: { id: productId } });

    if (product) {
      const reviews = await prisma.review.findMany({ where: { productId: product.id } });
      const averageRating = reviews.reduce((sum, review) => sum + review.rating, 0) / reviews.length;

      res.json({ ...product, reviews, averageRating });
    } else {
      res.status(404).json({ error: 'Product not found' });
    }
  } catch (error) {
    console.error('Error retrieving product:', error);
    res.status(500).json({ error: 'Internal Server Error' });
  }
});
```

- a. This line applies decimal parsing to `req.params.id`. If the review is not found, the server will respond with a 200 status.
- b. This line converts `req.params.id` to an integer. If the review is not found, it will cause a runtime error and crash.
- c. This line applies decimal parsing to `req.params.id`. If the review is not found, the server will respond with a 500 error status.
- d. This line converts `req.params.id` to an integer. If the review is not found, the server will response with a 500 error status.

23. Based on the three tables, select the answer below to get the names of students who have scores above the average for all courses taken?

```
CREATE TABLE Students (  
  student_id INT PRIMARY KEY,  
  student_name VARCHAR(255) NOT NULL,  
  date_of_birth DATE NOT NULL  
);  
  
CREATE TABLE Courses (  
  course_id INT PRIMARY KEY,  
  course_name VARCHAR(255) NOT NULL  
);  
  
CREATE TABLE Enrollments (  
  enrollment_id INT PRIMARY KEY,  
  student_id INT,  
  course_id INT,  
  grade DECIMAL(3, 2),  
  FOREIGN KEY (student_id) REFERENCES Students(student_id),  
  FOREIGN KEY (course_id) REFERENCES Courses(course_id)  
);
```

- a. `SELECT s.student_name FROM Students s INNER JOIN Enrollments e ON s.student_id = e.student_id INNER JOIN Courses c ON e.course_id = c.course_id WHERE e.grade > (SELECT AVG(grade) FROM Enrollments GROUP BY student_id);`
- b. `SELECT s.student_name FROM Students s INNER JOIN Enrollments e ON s.student_id = e.student_id INNER JOIN Courses c ON e.course_id = c.course_id WHERE e.grade > (SELECT AVG(grade) FROM Enrollments GROUP BY course_id);`
- c. `SELECT s.student_name FROM Students s INNER JOIN Enrollments e ON s.student_id = e.student_id INNER JOIN Courses c ON e.course_id = c.course_id GROUP BY s.student_name WHERE AVG(e.grade) > (SELECT AVG(grade) FROM Enrollments);`
- d. `SELECT DISTINCT s.student_name FROM Students s INNER JOIN Enrollments e ON s.student_id = e.student_id INNER JOIN Courses c ON e.course_id = c.course_id WHERE e.grade > (SELECT AVG(grade) FROM Enrollments);`